

CODEJENN: A simple C++ neural net generator

Jay Arcities^a, Eric Ching^b, Kamal Viswanath^b, Joshua E. Lipman^c, Kevin Dong^c, Pavel Popov^a, Hai Wang^c, Ryan Johnson^b

^a*Aerospace Engineering Department, San Diego State University, 5500 Campanile Drive, San Diego, 92182, CA, USA*

^b*Laboratories of Computational Physics, Naval Research Lab, 4555 Overlook Ave SW, Washington, 20375, DC, USA*

^c*Department of Mechanical Engineering, Stanford University, Stanford, CA, 94305, USA*

Abstract

Machine intelligence is a rapidly growing field and becoming state-of-the-art in improving numerical methods for computational physics. This has enabled researchers to perform work in data-driven preconditioners [1] [2], learning bases surrogate constitutive models [3], and physics informed neural nets [4]. Although libraries like PyTorch [5], TENSORFLOW-KERAS [6], and ONNX [7] make machine learning development and implementation straightforward, many researchers are bound to only Python. This becomes a limitation when performance is crucial, since many high-fidelity physics solvers are implemented in low-level languages like C++. We introduce CODEJENN, a high-performance library that bridges this gap by taking a trained TENSORFLOW-KERAS neural network and auto-generating fully self-contained C++ code. The library’s implementation integrates smoothly into existing high-fidelity physics solvers via inlined functions, eliminating external dependencies and preserving the model’s predictive capability. In contrast to the C++ APIs exposed by these Python-based libraries, CodeJeNN produces lightweight, standalone code that scales efficiently on local architectures. We demonstrate CodeJeNN on a set of computational-fluid-dynamics (CFD) test cases, by training neural nets, outlining the code-generation workflow, and validating the generated solvers against reference solutions.

Keywords: Deep Learning, Computational Physics, Inference, Code Generation, Neural Networks

1. Introduction

Machine learning has gained significant traction and has shown a lot of promise when integrated with numerical schemes for solving partial differential equations (PDEs). This trend of specifically using neural nets has been attributed to the *Universal Approximation Theorem* (UAT) that states that neural nets with sufficient size, in principle, approximate any continuous function to any desired degree of accuracy. More specifically, in computational fluid dynamics (CFD), H. Wang et al. [8] review how neural nets have been applied in diverse contexts, including reduced-order modeling, adaptive meshing, and direct solution of governing PDEs via physics-informed neural nets (PINNs). They also categorize different architectures, such as multi-layer perceptrons (MLPs), convolutional neural nets (CNNs), fourier transformers, and graph neural nets (GNNs)—and explain how each is applied.

Each neural net architecture brings different strengths to the table, giving computational physics practitioners the flexibility to test and implement various surrogate learning models using high-level libraries like ONNX, TensorFlow, PyTorch, and others. In practice, however, speed and accuracy are critical during inference. Which is one reason many physics solvers are still written in low-level languages such as fortran, C++, etc. Since most state-of-the-art machine learning libraries are written in Python, integrating ML surrogate models into existing CFD code often adds overhead. Plus, machine learning is still a relatively new field for researchers in computational science, and diving into the math and implementation details can be a steep time and energy investment. Some neural net architectures are overly complex, and even then, there is no guarantee they are suitable for a given problem, meaning more time spent experimenting and coding up alternatives.

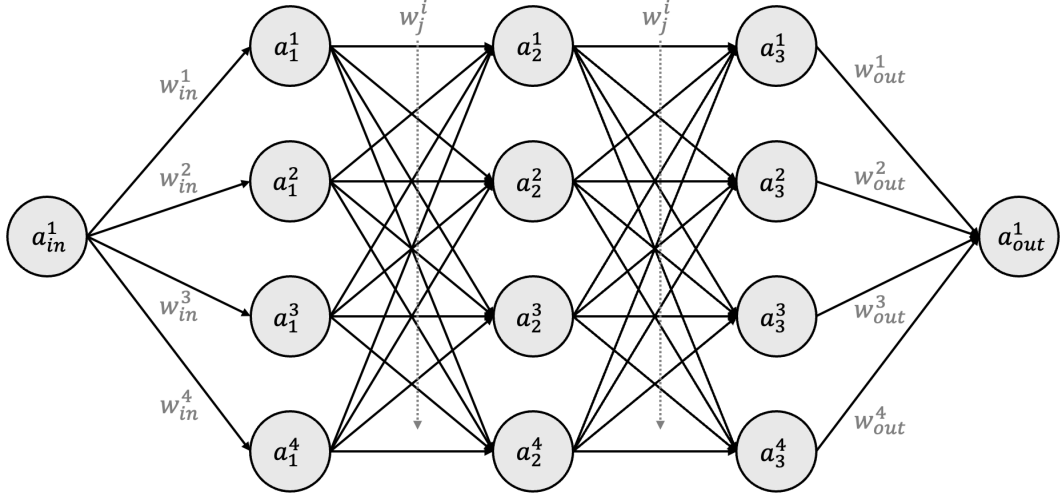


Figure 1: Schematic of a feedforward network (MLP), where a is the output from that node, w_j^i is the specific weight sent to that nodes given the edge.

Because of the gap between high level language learning libraries integrated into low-level language physics solvers, researchers have resorted to using these machine learning libraries C++ API's as a way to incorporate these surrogate learning models into their codebase. The main issue with this is that many physics solvers are typically parallelized and vectorized. Outsourcing functions to external libraries may not allow users to optimize their code efficiently and may increase costs due to the mismanagement of computing resources. Bati and Bryngelson [9] have developed a FORTRAN-based encoder, where they generate code for neural nets, specifically MLPs and RNNs. This is a great alternative for researchers who do not want to go the C++ API route for their codebase.

To expand upon that, we present CODEJENN, a C++ based encoder that code generates an a priori trained neural nets created developed in TENSORFLOW-KERAS. In this library, we also generate all supporting functions in plain C++. This allows users to retain raw source code and optimize at their discretion rather than depending on external libraries such as LAPACK, CVODES, or EIGEN. This paper will outline the perspective used to model neural net inference, how CODEJENN code generates based off that perspective, about the machine learning library CODEJENN is compatible with, the implementation and limitations of CODEJENN, and the usage and example cases.

2. Neural Net Principles

A neural net is a surrogate model used in deep learning to predict the outcome of a data set given some initial data. The most common neural net architecture is an MLP, where during inference, each layer propagates information from input to output. The way it does that is by an affine transformation with perturbation followed by an activation function to introduce non-linearity as shown in Eq. 1. Moreover, a typical schematic of an arbitrary MLP is shows in Fig. 1

$$\mathbf{a} = \sigma(\mathbf{w}^T \mathbf{x} + \mathbf{b}) \quad (1)$$

Of course, other architectures such as GNN and CNN, have extended renditions of this equation where, for example, a CNN filters out spatial relations from a 2D matrix input data sample as shown in Eq. 2. Where c_{out} and c_{in} are the input and output channel index of the filter, m and n are the spatial index in the filter, i and j is the index of the input data, and K is the filter.

$$Y_{c_{out},i,j} = \sum_{c_{in}} \sum_m \sum_n K_{c_{out},c_{in},m,n} X_{c_{in},i+m,j+n} + b_{c_{out}} \quad (2)$$

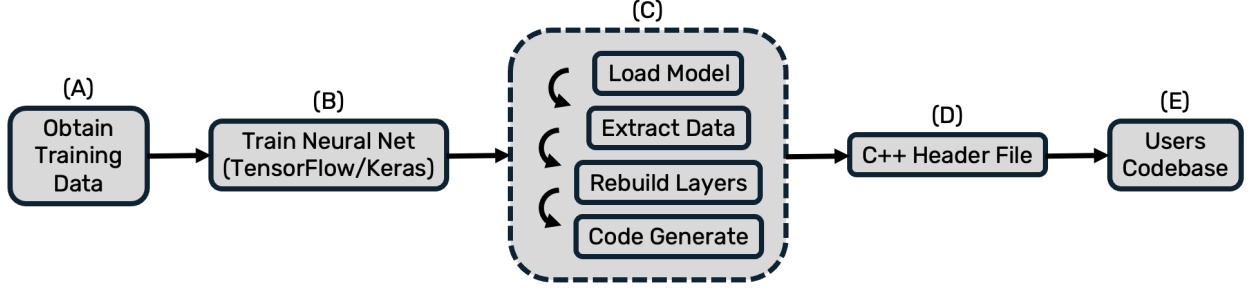


Figure 2: CodeJeNN (C) code generates a KERAS trained neural net into a C++ header file for the users computational physics solver (E).

The main principle CodeJeNN utilizes is that these neural nets are made up of layers where information passes through from input to output. This property is shared through all neural net architectures. Similarly to how a neural architecture is built, CodeJeNN propagates through each layer extracting all important parameters that are used for inference, and rebuilds that structure in a self-contained C++ header file.

more to add

3. CodeJeNN Architecture

Figure 2 shows the flowchart of how CodeJeNN works. First, the user obtains well-posed data samples to train a neural net on. Next, the user trains the model based on the machine learning library KERAS, which is integrated into TENSORFLOW. The model architecture to which the user can build does have limitations but is very extensive. This is primarily based on the capabilities of CODEJENN. Next the user will save that trained model into a .KERAS or a .H5 file. Then CODEJENN will code generate a self sufficient neural net in C++. The principle of CodeJeNN is to inline as much as possible, scale down to local machine, and be self-sufficient using in-house functions. So far, CodeJeNN can work on MLP's and CNN's but will soon extend to GNN's.

3.1. TensorFlow-Keras Compatibility

There are numerous python based machine learning libraries used that have sufficient API for C++ applications. However, they may not scale the same way when used within already highly optimized computational physics codes. In these codes, it can be assumed that every computational method for speed up is taken, leading to no threads to be used left, in turn harming performance. The library that CODEJENN is compatible with is KERAS which is built into TENSORFLOW

At the time of building CODEJENN, KERAS was chosen because it saves all hyperparameters (weights and biases), training configurations (optimizers, loss), and the model's architecture all in a single file. This allows users to pass and receive a single file at will with all information and code generate easily. PYTORCH is also very great choice of training a model, and is commonly used. However, PYTORCH only saves the models weights as a dictionary. This way, users would have to hardcode their model architecture in their codebase when loading the model. This is crucial as, many researchers want a simple workflow of grabbing data, training models, and code generated a C++ neural net on the fly. Thus, KERAS was the library of choice.

3.2. Code Generating

There are two main ways a model can be built. The first is by using the sequential model class, Algorithm 1, where layers are stacked on top of one another. The other is using the functional model class, Algorithm 2 where each layer output is an input for the next layer. In either case, it is important that the model is built where it only accepts a single input and output data set of any dimensions of any size. Then, CODEJENN has four main steps to code generate.

Algorithm 1 MLP built using KERAS sequential model class

```
1  model = models.Sequential([
2      # Layer 1
3      layers.Input(shape=(INPUT_DIM,)),
4      layers.LayerNormalization(),
5      layers.LeakyReLU(negative_slope=0.01),
6      # Layer 2
7      layers.Dense(NEURONS, activation=None),
8      layers.Activation('relu'),
9      # Layer 3
10     layers.Dense(NEURONS, activation=None),
11     layers.Activation('mish'),
12     # Output
13     layers.Dense(OUTPUT_DIM, activation=None)
14 ])
```

Algorithm 2 MLP built using KERAS functional model class

```
1  inputs = layers.Input(shape=(INPUT_DIM,))
2  # Layer 1
3  x = layers.Dense(NEURONS, activation=None)(inputs)
4  x = layers.LayerNormalization()(x)
5  x = layers.LeakyReLU(negative_slope=0.01)(x)
6  # Layer 2
7  x = layers.Dense(NEURONS, activation=None)(x)
8  x = layers.Activation('relu')(x)
9  # Layer 3
10 x = layers.Dense(NEURONS, activation=None)(x)
11 x = layers.Activation('mish')(x)
12 # Output
13 outputs = layers.Dense(OUTPUT_DIM, activation=None)(x)
14 model = models.Model(inputs, outputs)
```

First, the model is loaded into a model variable, to which, options can be appended to it, such as compiling the model if further training is needed. Or, for instance, if a serialized custom activation function is saved in the model, where the user can code generate a custom activation function. Moreover, this also allows models that were trained from different versions and distributions of TENSORFLOW-KERAS to still load properly. This allows for the user to also add new API into CODEJENN if they wish to load more advanced models with numerous options. Second, the model is extracted of all parameters based on the architecture. Each layer is propagated through saving each parameter in a dictionary. This is where the user, if need be, can expand if there is a certain layer that is not included into the API. It is also noted that each layer itself can have numerous options and configurations. Third, the model is rebuilt from the input layer to the output layer, where then on step 4, the model is code generated into C++ in a header file ready to be used.

Algorithm 3, shows a detailed schematic of how CODEJENN formats the generated model. First the forward pass functions (similar to edges of in Figure 1, where we go from one layer to the next) are defined before the actual MLP function. Then within the models function, each layer parameters are generated, then the activation functions are defined in the models function as lambda functions, finally the forward pass function are printed, where inference is computed. CODEJENN code generates neural nets in this format for easier readability, implementation, and computing cache space. More information on why can be found in Section 3.3.

Algorithm 3 Format of an example code generated MLP

```

1: Import packages: iostream, array, cmath, functional, etc.

2: Define forward pass functions:
3:   Dense(outputs, inputs, weights, biases, input size, activation)
4:   UnitNormalization(outputs, inputs, epsilon)

5: function MLP(input)
6:   Print model parameters:
7:     Layer weights and biases
8:     Filter parameters
9:     Input/Output normalization constants

10:  Define activation functions:
11:    linear(input)
12:    gelu(input)
13:    custom_activation(input, index)

14:  Forward pass:
15:    Normalize  $x \leftarrow (x - \mu_{in})/\sigma_{in}$ 
16:     $h_1 \leftarrow \text{Dense}(x, W_1, b_1, \text{linear})$ 
17:     $h_2 \leftarrow \text{UnitNormalization}(h_1, \epsilon)$ 
18:     $h_3 \leftarrow \text{gelu}(h_2)$ 
19:     $h_4 \leftarrow \text{Dense}(h_3, W_4, b_4, \text{linear})$ 
20:     $h_5 \leftarrow \text{custom\_activation}(h_4)$ 
21:    Denormalize:  $y \leftarrow h_5 \cdot \sigma_{out} + \mu_{out}$ 
22:  return  $y$ 
23: end function

```

3.3. Algorithm Structure Specification

In terms of layout and data structures, CODEJENN uses constant pre-initialized static arrays. As a result, all functions are void and inlined, directly manipulating the data structure itself. It is written so that everything is known at compile time, allowing efficient use of cache space and fast inlined computation. It's worth noting that variadic templating, as opposed to for-loops, was considered. However, given an

MLP with an input data sample size of 1×400 and a first hidden layer of 256 nodes, that would require 102,400 weights just for the first layer. This would quickly overwhelm cache space and lead to compilation issues. Another important point is that every syntax-based speedup is explicitly applied. For example, `void`, `inline`, `constexpr`, and `static` are consistently used across arrays, headers, and function templates. This is the primary optimization taken. Reducing computational cost is the top priority, so CODEJENN is inlined as much as possible. The generated neural net is raw source code, matching the exact implementation of the model it was derived from. This lets users further optimize the source code to their own needs. Many computational physics codes are already heavily optimized, so having CODEJENN aggressively optimized on its own, and then integrated, can actually hurt performance. Because of this, compile-based optimizations—and even outsourcing functions to external libraries—are left up to the user. This is also why forward-pass functions are defined at the top: they benefit the most from user-specific optimization. Additionally, within the lambda activation functions, as well as the function calls near the bottom, there may be for-loops that can be easily parallelized or vectorized.

3.4. Capabilities and Limitations

So far CODEJENN supports the following activation functions

ReLU	LINEAR	GELU
SIGMOID	ELU	SOFTMAX
TANH	SWISH/SiLU	MISH
LEAKYReLU	SiLU	SOFTPLUS

In addition, CODEJENN also supports custom activation functions. The only required steps is for (1) the user to serialize their custom activation function when saving their model, and (2) when loading it, define it in a python script. An example case of how to define, save, and code generate using a custom activation function is shown in [10].

With how we defined neural nets to be a stack of different types of layers, CODEJENN supports the following layers from KERAS layer API [6]:

- Dense Layers (pure MLP)
- Pure Activation Layers
- Dropout, and 1D, 2D, and 3D Spatial Dropout Layers [11]
- Reshape, Flatten, and Rescale Layers
- Layer, Batch, Group, and Unit Normalization Layers
- 1D, 2D, and 3D Convolutional Layers
- 1D, 2D, and 3D Transposed Convolutional Layers [12] [13]
- 1D and 2D Depthwise Convolutional Layers
- 1D and 2D Seperable Convolutional Layers
- 1D, 2D, and 3D Max and Average Pooling Layers
- 1D, 2D, and 3D Global Max and Global Average Pooling Layers

Training a neural net might not be guaranteed to converge, thus CODEJENN also supports normalization parameters for the input and output of the neural net. Again, an example case is shown in [10]. For inference, because CODEJENN deals with row-major flat arrays only, flatten layers have no effect on the code generation process but are there to catch layers that have models serialized with flatten layers. Similarly, dropout layers do not have dedicated layers but it does affect inference through a given layer’s weights. Depending on the dropout factor, certain weights would just read 0.0.

There are limitations to CODEJENN as well. The main one being that CODEJENN only supports single branch neural nets. Multi-input/output neural nets are not supported although the user can build models in KERAS that support that. Some additional limitations stem from how many layers CODEJENN can support. Many of these layers have numerous options that the user may add or want to configure. Because the code generation process can only pick up on so many of these options, it is in the best interest of the user to see how these layers’ parameters are extracted.

4. Results

A user friendly, intuitive neural net library has been demonstrated. **blah blah blah** With any accessible code, there is the inevitable obstacle of running into logic errors as well. Thus CODEJENN also has a debugging API meant to help users understand where their issue is. CODEJENN has a very extensive API for debugging. KERAS has a model summary of a model that has been built and train, and CODEJENN compares what was extracted from that model to what KERAS model summary gives. Additionally, even when the C++ model compiles and show no compiling issues, there can still be logic/algebraic errors. With the debug flag on, a print statement at the end of each layer is also generated printing the first 10 values of the output of a given layer. This allows the user to pin point errors in their model. It has been shown from testing, the majority of issues arise from user custom activation function. Where the users python custom activation function does not match its C++ counterpart.

4.1. Chemistry in a detonation wave

Accurately simulating detonations is becoming critical to the design of detonation-based engines, a critical technology for the next generation of propulsion and power generation devices. Solving detailed chemistry in detonation simulations is physically accurate, but requires immense computational expense, especially in simulations with large chemical kinetic mechanisms [? ?]. This section aims to demonstrate the application of CodeJeNN to circumvent solving a stiff set of ordinary differential equations (ODEs) for each species production rate in the unsteady, compressible, multicomponent reacting flow solver detonationFoam. detonationFoam is chosen for a few key reasons. Namely, it is open-source, adaptive mesh refinement (AMR) capable, and has a demonstrated ability to accurately reproduce the cell size of the detonation case targeted in this work [?]. AMR densifies grid points near the detonation front, providing enhanced resolution in these regions. For more details on detonatonFoam, refer to [?]. Within detonationFoam, each species' production rate is modeled as a linear function of the form

$$\omega_k = W_k * \frac{C_k^{i+1} - C_k^i}{dt} \quad (3)$$

where ω_k is the source term of species k , W_k is the molecular weight of species k , C_k^{i+1} is the molar species concentration of species k at time $i + 1$, C_k^i is the molar concentration of species k at time i , and dt is the duration of the timestep. Typically, C_k^{i+1} is derived from integrating a stiff set of ordinary differential equations formulated from a chemical kinetic mechanism. Therefore, the neural network aims to follow the same formulation by predicting the change in species concentration over the timestep.

To optimize prediction accuracy, the neural network is trained to correct a first-order approximation for the source term. More specifically, the first order approximation is obtained by evaluating the rate of progress, q_i , for each reaction i in a chemical kinetic mechanism with K species:

$$q_i = k_{fi} \prod_{k=1}^K [X_k]^{\nu'_{ki}} - k_{bi} \prod_{k=1}^K [X_k]^{\nu''_{ki}} \quad (4)$$

Here, k_{fi} is the forward reaction rate constant, k_{bi} is the backward reaction rate constant, $[X_k]$ is the molar concentration of species k , ν'_{ki} is the reactant stoichiometric coefficient of species k , and ν''_{ki} is the product stoichiometric coefficient of species k . The thermodynamic state is that of the current timestep in evaluating these rates of progress. The backward reaction rate can be evaluated from the forward reaction rate and the equilibrium constant, as explained in [?]. Therefore, the new source term is

$$\omega_{k,NN} = S_k \cdot dt + NN(\mathbf{C}, T, p) \quad (5)$$

$\omega_{k,NN}$ is the source term from the neural network formulation, S_k is the first order estimate derived from the rate of progress of each reaction, $NN(\mathbf{C}, T, p)$ is the neural network-predicted correction to the first order estimate. The neural network bases its prediction on the list of species concentrations $\mathbf{C}$, temperature T , and pressure P .

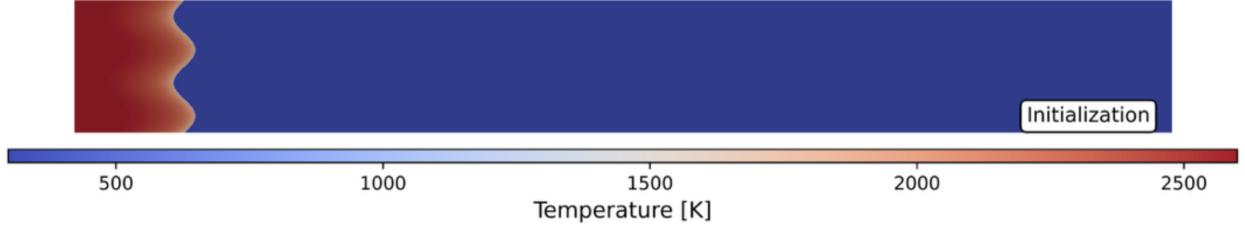


Figure 3: Temperature profile of ZND initialization.

The example case tested here is a low-pressure, stoichiometric hydrogen, oxygen, and argon detonation. This case is commonly used to verify numerical chemically reacting flow solvers, such as in Johnson & Kercher [?]. Numerical and experimental studies have established a cell size of approximately $(0.055) \text{ m} \times (0.03) \text{ m}$ for this case. The details of the case are listed here:

$$X_{Ar} : X_{H_2} : X_{O_2} = 7 : 2 : 1 \quad (6)$$

$$P_u = 6670 \text{ Pa} \quad (7)$$

$$T_u = 298 \text{ K} \quad (8)$$

where X_{Ar} , X_{O_2} , X_{H_2} are the mole fractions of argon, oxygen, and hydrogen, respectively. T_u is the unburned gas temperature, and P_u is the unburned gas pressure. To assess the accuracy of the neural network formulation, a baseline simulation with detailed finite-rate chemistry integration is necessary. The channel dimensions for this simulation are $(0.5) \text{ m}$ by $(0.06) \text{ m}$. The top and bottom boundary conditions are slip walls, and the front and back boundary conditions are zero gradient inflow and outflow. This simulation was performed with a 2000 by 240 computational grid and 3 levels of AMR. Thus, the smallest resolvable cell dimension is 31.25 micrometers by 31.25 micrometers. This resolution is selected due to its relatively inexpensive computational cost and ability to accurately reproduce the expected cell pattern for this test case. This case uses the Euler governing equations and the hydrogen sub-mechanism of FFCM2. Furthermore, the simulation has the initial condition of a sinusoidally perturbed Zeldovich - von Neumann - Döring (ZND) one-dimensional, steady, detonation. The ZND detonation is evaluated using the Caltech SDtoolbox and the previously mentioned chemical kinetic mechanism [?]. This initial condition is shown in Figure 3. The sinusoidal nature of the initial condition enables quick destabilization and formation of the desired two-dimensional detonation structure and propagation mechanism. After enough time, the detonation reaches a steady state where its propagation is independent of its initialization mechanism. A fixed timestep of 1 nanosecond is used.

The neural network is a fully-connected neural network, consisting of three hidden layers of size 1024 , 10 , and 1024 , respectively. Each hidden layer uses a ReLU activation function. The input size is 10 , consisting of temperature, pressure, and the species concentration of all species except the diluent argon. The output layer is also size 10 , consisting of the predicted change in temperature, pressure, and the corrections to the first-order approximation of the source term for all species except argon. Thermodynamic states for neural network training are collected from the baseline simulation. States are extracted every $1,000$ timesteps (1 microsecond) for the first $28,000$ simulation timesteps. At each timestep, all computational cells within 0.1 m of the leading shock front are extracted. This resulted in approximately 7 million training data points, which were naturally heavily concentrated near the detonation front due to use of AMR refinement based on density gradient magnitude.

The neural network output data is generated by performing a 1 -nanosecond, constant-volume, homogeneous reactor simulation in Cantera [?]. These simple simulations isolate the input thermodynamic state's evolution due to the chemistry. These simulations are solved using the same chemical kinetic mechanism mentioned above. The neural network is effectively learning the deviation from the first-order approximation of the source term for every thermodynamic input state over the course of 1 nanosecond. Training

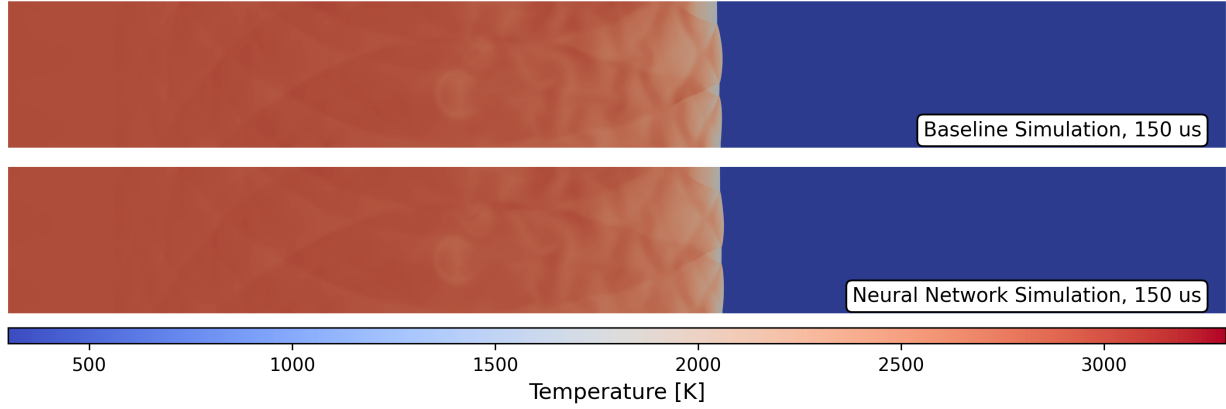


Figure 4: Temperature profile after 150 microseconds of simulation time.

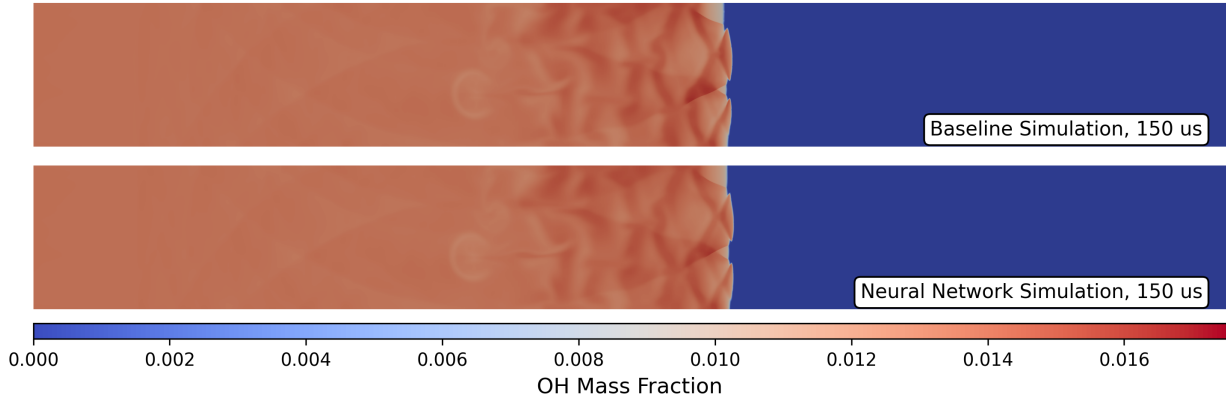


Figure 5: OH mass fraction profile after 150 microseconds of simulation time.

the neural network in this manner aligns with the overall goal of the neural network replacing chemical integration. Lastly, it is important to note that the neural network was trained with eight different learning rates. Learning rate is the neural network hyperparameter most sensitive to performance. The learning rate that produced the smallest validation loss is the one tested.

The results of this effort are shown in Figure ???. These figures demonstrate a comparison of the baseline simulation (top) and the neural network formulation (bottom) for temperature, OH mole fraction, and maximum pressure, respectively. Temperature evolution provides an overall metric to assess simulation accuracy. Maximum pressure history represents a numerical soot foil. OH is the most important radical in hydrogen combustion, dictating key chain-propagation and branching reactions. Together, these three features provide a platform for quantitative comparison between the baseline simulation and the neural network-based formulation. All results are from 150 microseconds (150,000 numerical timesteps) of simulation time. Lastly, it is important to acknowledge that the high level of accuracy in the results is largely a result of the nearly adequate first-order approximation. Nevertheless, this example demonstrates using CodeJeNN to seamlessly embed a neural network into a CFD solver.

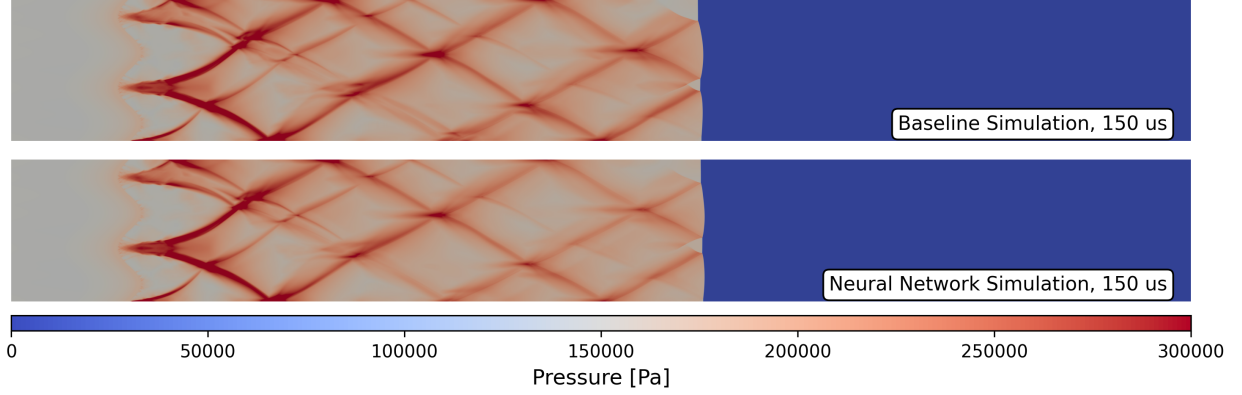


Figure 6: Maximum pressure time-history after 150 microseconds of simulation time.

5. Conclusion

We have developed a performant in-house C++ based neural net generator. This library is compatible with TENSORFLOW-KERAS, a popular deep learning library used in computational physics. We have demonstrated CODEJENN using a NN trained for chemistry and its usage in a test detonation example case. Future works include full integration of well-used GNN layers, and in-depth tutorials in their usage for matrix manipulation. Future work will also explore accelerating code generated neural net by adding options to export functions using low-level high performant libraries.

Appendix A. Example Appendix Section

References

- [1] P. Häusner, A. N. Juscafresa, J. Sjölund, Learning incomplete factorization preconditioners for GMRES, arXiv:2409.08262 [cs] (Dec. 2024). doi:10.48550/arXiv.2409.08262.
URL <http://arxiv.org/abs/2409.08262>
- [2] Y. Li, P. Y. Chen, T. Du, W. Matusik, Learning Preconditioners for Conjugate Gradient PDE Solvers.
- [3] J. N. Fuhg, G. A. Padmanabha, N. Bouklas, B. Bahmani, W. Sun, N. N. Vlassis, M. Flaschel, P. Carrara, L. D. Lorenzis, A review on data-driven constitutive laws for solids, arXiv:2405.03658 [cs] (May 2024). doi:10.48550/arXiv.2405.03658.
URL <http://arxiv.org/abs/2405.03658>
- [4] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics Informed Deep Learning (Part I): Data-driven Solutions of Nonlinear Partial Differential Equations, arXiv:1711.10561 [cs] (Nov. 2017). doi:10.48550/arXiv.1711.10561.
URL <http://arxiv.org/abs/1711.10561>
- [5] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews,

- G. Chanan, P. Wu, S. Chintala, PyTorch 2: Faster Machine Learning Through Dynamic Python Byte-code Transformation and Graph Compilation, in: 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), ACM, 2024. doi:10.1145/3620665.3640366.
URL <https://docs.pytorch.org/assets/pytorch2-2.pdf>
- [6] F. Chollet, et al., Keras, <https://keras.io> (2015).
- [7] J. Bai, F. Lu, K. Zhang, et al., ONNX: Open Neural Network Exchange, <https://github.com/onnx/onnx> (2019).
- [8] H. Wang, Y. Cao, Z. Huang, Y. Liu, P. Hu, X. Luo, Z. Song, W. Zhao, J. Liu, J. Sun, S. Zhang, L. Wei, Y. Wang, T. Wu, Z.-M. Ma, Y. Sun, Recent advances on machine learning for computational fluid dynamics: A survey. *arXiv:2408.12171 [cs]*, doi:10.48550/arXiv.2408.12171.
URL <http://arxiv.org/abs/2408.12171>
- [9] A. Bati, S. H. Bryngelson, RoseNNa: A performant, portable library for neural network inference with application to computational fluid dynamics, *Computer Physics Communications* 296 (2024) 109052, *arXiv:2307.16322 [physics]*. doi:10.1016/j.cpc.2023.109052.
URL <http://arxiv.org/abs/2307.16322>
- [10] J. Arcities, CodeJeNN (Sep. 2025).
URL <https://github.com/jarcities/CodeJeNN>
- [11] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, C. Bregler, Efficient Object Localization Using Convolutional Networks, *arXiv:1411.4280 [cs]* (Jun. 2015). doi:10.48550/arXiv.1411.4280.
URL <http://arxiv.org/abs/1411.4280>
- [12] V. Dumoulin, F. Visin, A guide to convolution arithmetic for deep learning, *arXiv:1603.07285 [stat]* (Jan. 2018). doi:10.48550/arXiv.1603.07285.
URL <http://arxiv.org/abs/1603.07285>
- [13] M. D. Zeiler, D. Krishnan, G. W. Taylor, R. Fergus, Deconvolutional networks, in: 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, IEEE, San Francisco, CA, USA, 2010, pp. 2528–2535. doi:10.1109/CVPR.2010.5539957.
URL <http://ieeexplore.ieee.org/document/5539957/>